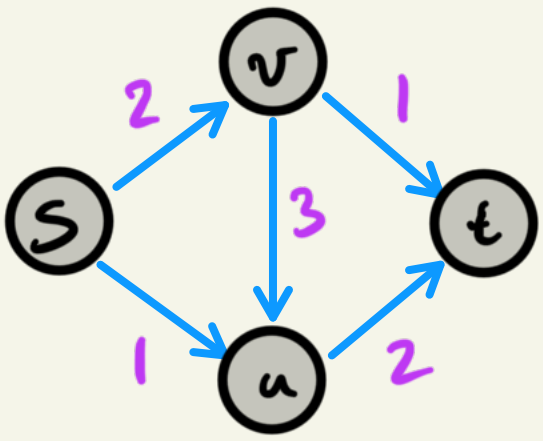
Network flows : MAX-FLOW MIN-CUT

Ford-Fulkerson Algorithm

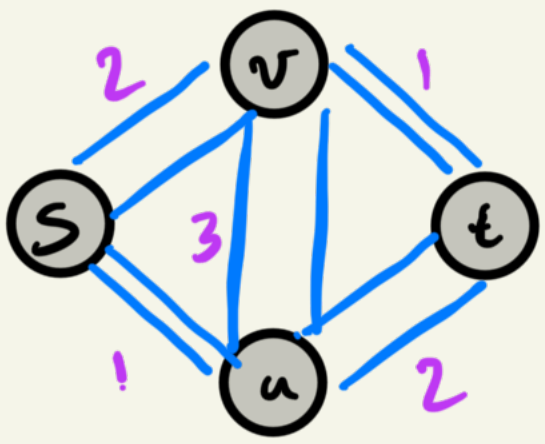
- Given $G = (V, E)$



CAPACITY c_e

indeg (s) = 0 source

outdeg (t) = 0 sink



Restriction

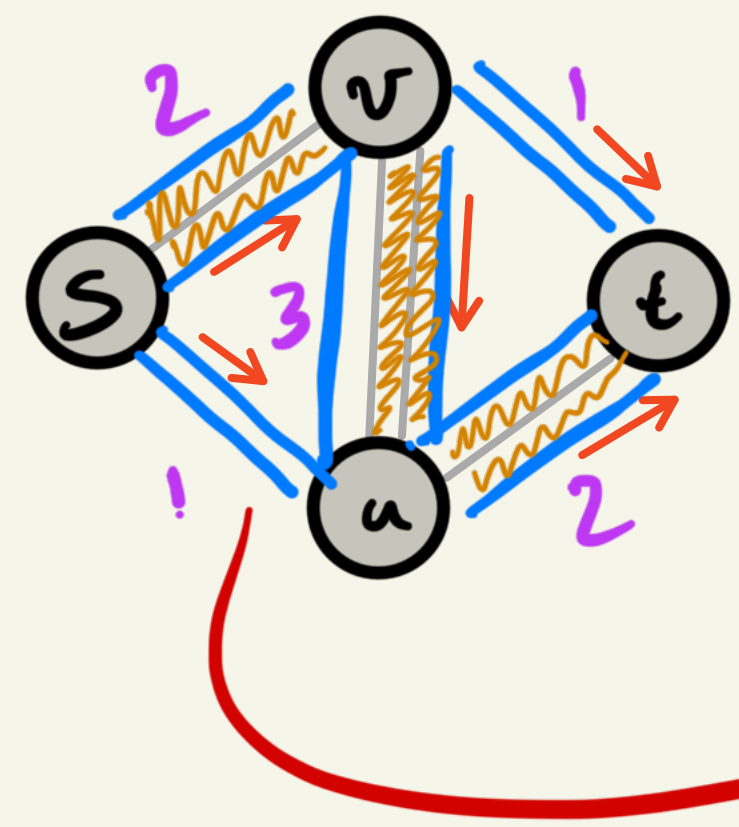
$0 \leq f(e) \leq c_e$

value of flow

$$v(f) = flow_{out}(s)$$

Max flow is limited to the minimum "cut" that we can find in the flow network

- Tentative :
  - greedy approach : push as much as possible edge coming out from the start

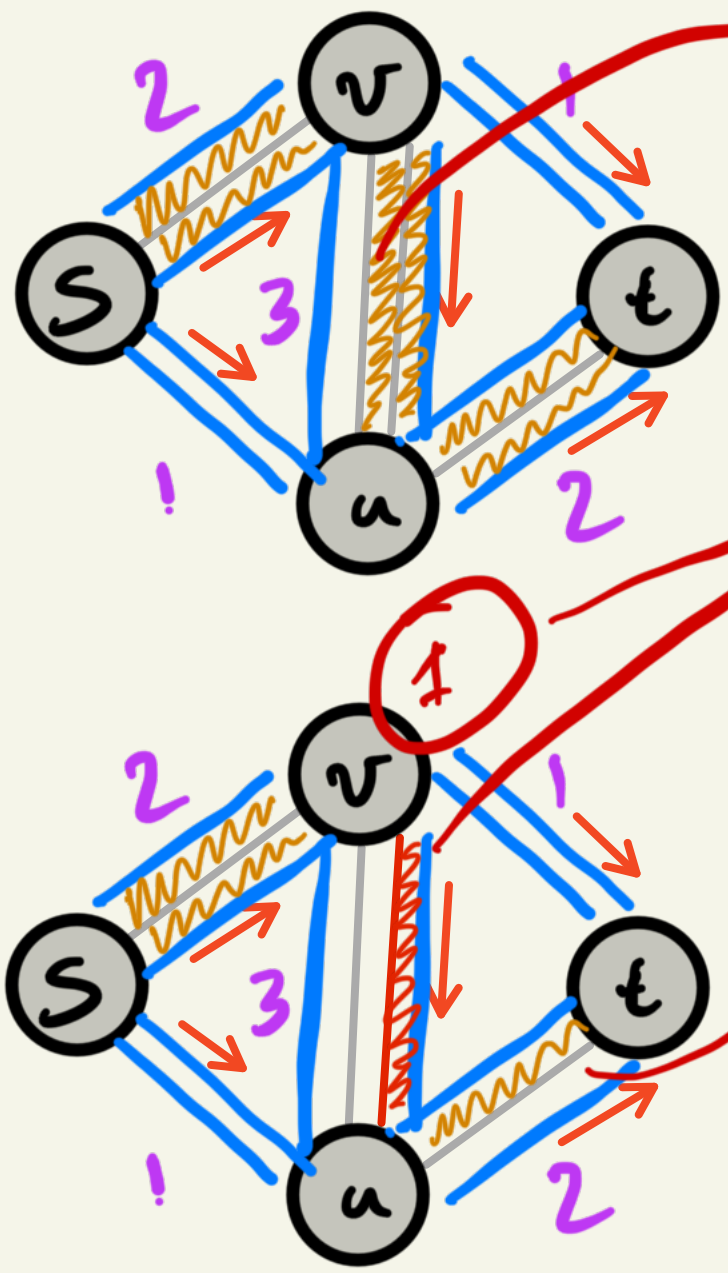


- If K units of flow reach a vertex u, then K units of flow must leave node u

- Value of flow $V(f) = 2$

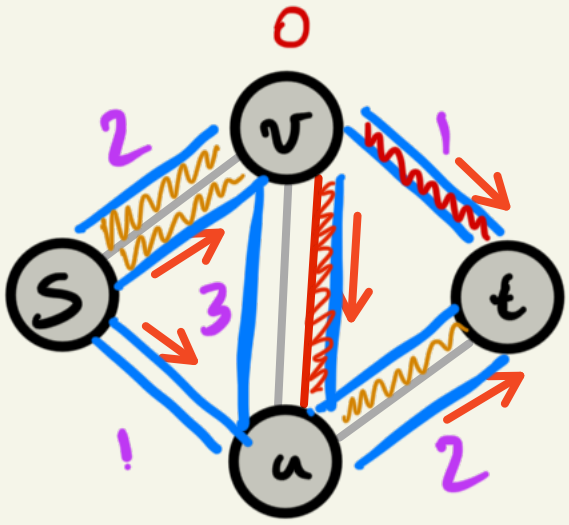
- Have we maximized that value?

still capacity here!
but here the flow will pass through?



- changing the flow here
- reducing flow on (v, u) to 1
- by conservation we need to reroute 1 flow unit at v

- Free 1 unit of flow at (u, t)



- Now we can push +1 flow out of s

$V(f) = 3$

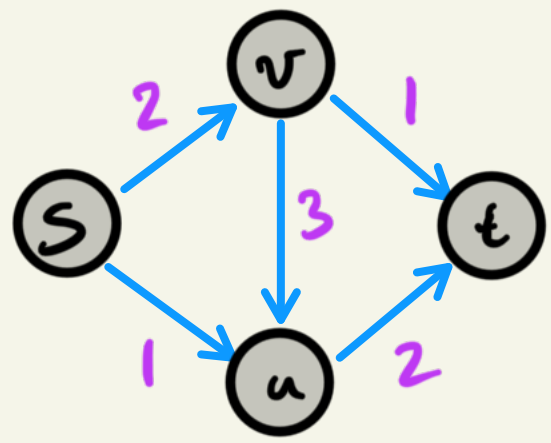
- this is the maximum flow that we can push to G

- the Ford-Fulkerson algorithm is a way to systemitize the process to find max-flow

- the proves ref. Cormen book

- basic Definition

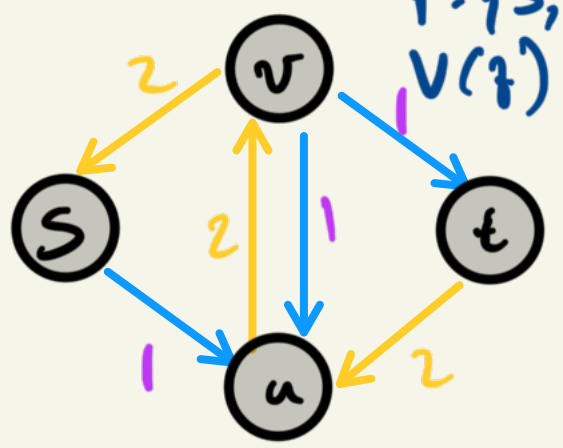
- Original graph



(no flow)

- Residual graph G'

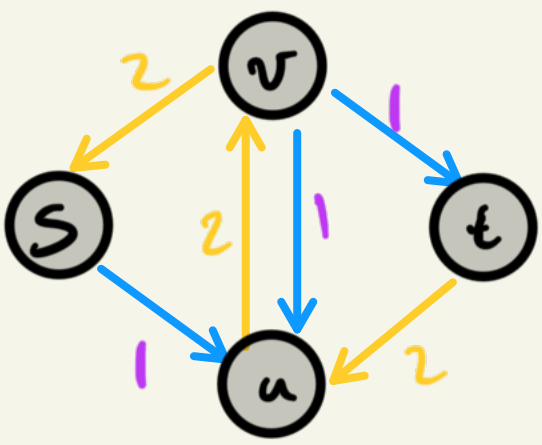
P: {s, v, u, t}
V(f) = 2



(with flow)

- this graph "says" where we can undo flow and where we can push more flow

- Next steps to find MAX-flow



$P' = \{ S, u, v, t \}$

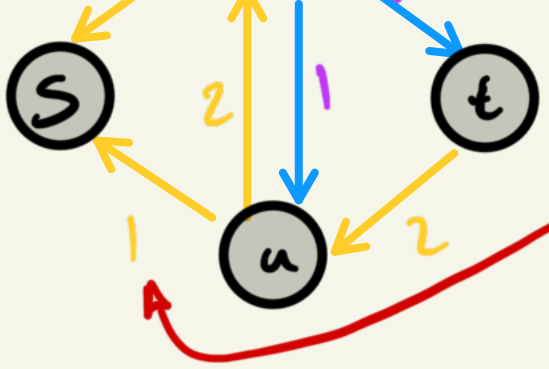
- increase S - u



$P' = \{ S, u, v, t \}$

- Note : Exist a path from s to t in the residual G', so it is possible to augument the flow

$P' = \{ S, u, v, t \}$

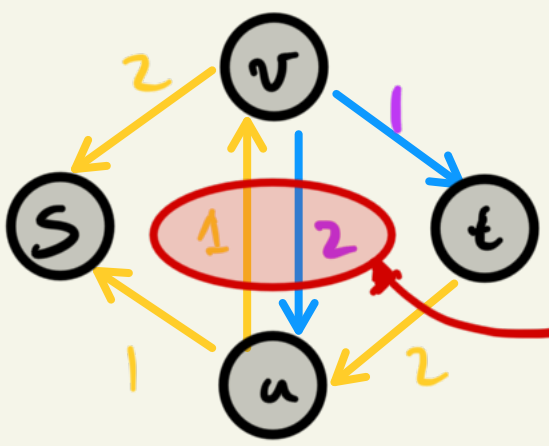


- G' says that

① I can push +1 flow to S

- Fix "undo" flow u-v

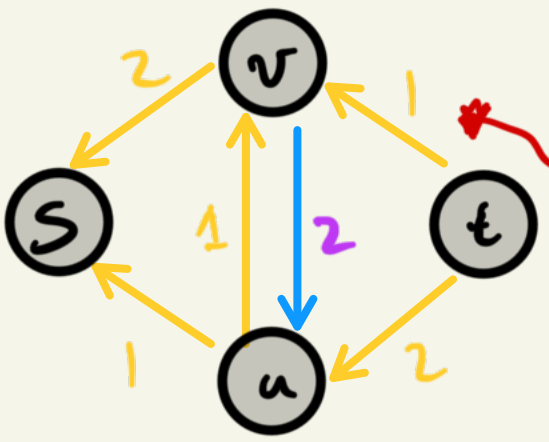
$P' = \{ s, u, v, t \}$



② there is a way to redirect the current flow between u-v to increase V(f)

- by conservation push flow v-t

$P' = \{ s, u, v, t \}$



$V(f) = 3$

③ what are the critical edges to look at to maximize V(f)

- Pseudo code

```
FF(G, s, t)
    for each (u,v) ∈ G
        flow[u][v] ← 0 ; flow[v][u] ← 0

    while ∃ path P s.t augument on G_res
        bottleneck ← ∞
        for (u,v) in P
          residual_capacity ← capacity[u][v] - flow[u][v]
          bottleneck ← min(bottleneck, residual_capacity)
```

```
for (u, v) in P
    flow[u][v] ← flow[u][v] + bottleneck
    flow[v][u] ← flow[v][u] − bottleneck

max_flow ← 0
for v adj to s:
    max_flow ← max_flow + flow[s][v]

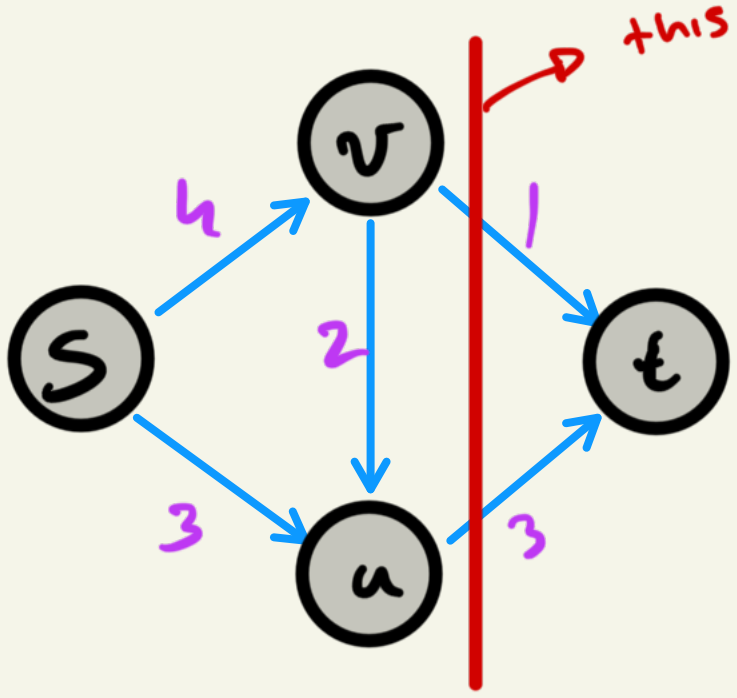
return max_flow
```

- What is "min-cut"?
  - we want a "cut" in the graph that split s and t in two different groups GROUP A and GROUP B of vertexs

$$A \cup B = V$$

A: 2 GROUPS; V: VERTEXS

$$A \cap B = \emptyset$$



- the smallest capacity cut will dictat the maximum flow in that net